॥वसुधैव कुटुम्बकम्॥

Symbiosis Institute of Technology, Pune

Faculty of Engineering

CSE - Academic Year 2026-27

Java Programming – Lab (Batch 2025-29)

Lab Assignment No:- 25

Name of Student	Abobakr Mohamed Elameen
PRN No.	25070122015
Batch	2025-29
Class	Programming with JAVA
Academic Year & Semester	2nd Year – Sem III
Title of Assignment	Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.
Practice questions	1. Create a program where multiple threads print numbers simultaneously.

```
package JAVA_EXP_25;
class NumberThread extends Thread {

    NumberThread(String name) {
        super(name);
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " : " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(getName() + " was
interrupted");
            }
        }
    }
}
```

```
public class MultiThreadNumbers {

    public static void main(String[] args) throws
InterruptedException {
        NumberThread t1 = new NumberThread("Thread-1");
        NumberThread t2 = new NumberThread("Thread-2");
        NumberThread t3 = new NumberThread("Thread-3");

        System.out.println("State of Thread-1 after creation    : "
+ t1.getState());

        t1.start();
        t2.start();
        t3.start();

        System.out.println("State of Thread-1 after start()     : "
+ t1.getState());

        Thread.sleep(200);
        System.out.println("State of Thread-1 during sleep()    : "
+ t1.getState());

        t1.join();
        t2.join();
        t3.join();

        System.out.println("State of Thread-1 after completion : "
+ t1.getState());
        System.out.println("All threads have finished
execution.");
    }
}
```

2. Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously.

```
package JAVA_EXP_25;
class Account {
    private int balance;

    Account(int balance) {
        this.balance = balance;
    }

    synchronized void deposit(int amount) {
```

```
        balance += amount;
        System.out.println(Thread.currentThread().getName() + "
deposited " + amount + " | Balance: " + balance);
    }

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println(Thread.currentThread().getName() +
" withdrew " + amount + " | Balance: " + balance);
        } else {
            System.out.println(Thread.currentThread().getName() +
" failed to withdraw " + amount + " | Insufficient balance: " +
balance);
        }
    }

    synchronized int getBalance() {
        return balance;
    }
}

class DepositThread extends Thread {
    private Account account;

    DepositThread(Account account, String name) {
        super(name);
        this.account = account;
    }

    public void run() {
        int[] amounts = {1000, 2000, 1500, 500};
        for (int amount : amounts) {
            account.deposit(amount);
            try {
                Thread.sleep(400);
            } catch (InterruptedException e) {
                System.out.println(getName() + " was
interrupted");
            }
        }
    }
}

class WithdrawThread extends Thread {
    private Account account;

    WithdrawThread(Account account, String name) {
        super(name);
```

```
        this.account = account;
    }

    public void run() {
        int[] amounts = {800, 3000, 1200, 6000};
        for (int amount : amounts) {
            account.withdraw(amount);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(getName() + " was
interrupted");
            }
        }
    }
}

public class BankTransactions {

    public static void main(String[] args) throws
InterruptedException {
        Account account = new Account(2000);
        System.out.println("Initial Balance: " +
account.getBalance());

        DepositThread depositThread = new DepositThread(account,
"Deposit-Thread");
        WithdrawThread withdrawThread = new
WithdrawThread(account, "Withdraw-Thread");

        System.out.println("Deposit thread state  : " +
depositThread.getState());
        System.out.println("Withdraw thread state : " +
withdrawThread.getState());

        depositThread.start();
        withdrawThread.start();

        depositThread.join();
        withdrawThread.join();

        System.out.println("Deposit thread state  : " +
depositThread.getState());
        System.out.println("Withdraw thread state : " +
withdrawThread.getState());
        System.out.println("Final Balance: " +
account.getBalance());
    }
}
```

Output	1) State of Thread-1 after creation : NEW State of Thread-1 after start() : RUNNABLE Thread-3 : 1 Thread-1 : 1 Thread-2 : 1 State of Thread-1 during sleep() : TIMED_WAITING Thread-2 : 2 Thread-1 : 2 Thread-3 : 2 Thread-3 : 3 Thread-1 : 3 Thread-2 : 3 Thread-3 : 4 Thread-1 : 4 Thread-2 : 4 Thread-2 : 5 Thread-1 : 5 Thread-3 : 5 State of Thread-1 after completion : TERMINATED All threads have finished execution. 2) a\jdt_ws\JAVA_PRACTICALS_eabe1187\bin' 'JAVA_EXP_25.BankTransactions' Initial Balance: 2000 Deposit thread state : NEW Withdraw thread state : NEW Deposit-Thread deposited 1000 | Balance: 3000 Withdraw-Thread withdrew 800 | Balance: 2200 Deposit-Thread deposited 2000 | Balance: 4200 Withdraw-Thread withdrew 3000 | Balance: 1200 Deposit-Thread deposited 1500 | Balance: 2700 Withdraw-Thread withdrew 1200 | Balance: 1500 Deposit-Thread deposited 500 | Balance: 2000 Withdraw-Thread failed to withdraw 6000 | Insufficient balance: 2000 Deposit thread state : TERMINATED Withdraw thread state : TERMINATED Final Balance: 2000
Conclusion	Thus, we have successfully created and managed multiple threads in Java using the Thread class. The programs show concurrent execution of threads and the stages of their lifecycle, and the bank transaction example shows how threads can safely share a common resource.